```
    requested = false
    httpRequest = (url) -> requested = url
    comments = new Comments 'http://the-url', {}, httpRequest
    requested.shouldBe "http://the-url/comments"

  it 'should bind to event on the element', ->
    comments = new Comments 'http://the-url', {}, ->
    element =
      querySelector: -> element
      value: 'A comment from Scruffy'

    comments.bind element, 'post'
    postReceived = false
    comments.post = (comment) -> postReceived = comment
    element.onpost()
    postReceived.shouldBe element.value

  it 'should render comments to the page as a list', ->
    out = innerHTML: (content) -> renderedContent = content
    comments = new Comments 'http://the-url', out, ->
    comments.render '["One", "Two", "Three"]'
    out.innerHTML.shouldBe "<ul><li>One</li><li>Two</li>
<li>Three</li></ul>"
```

The next listing contains the `Comments` module.

Listing 12.16. The `Comments` module (client/comments.coffee)

```
class Comments
  constructor: (@url, @out, @httpRequest) ->
    @httpRequest "#{@url}/comments", @render

  post: (comment) ->
    @httpRequest "#{@url}/comments?insert=#{comment}", @render
  bind: (element, event) ->
    comment = element.querySelector 'textarea'
    element["on#{event}"] =>
      @post comment.value
      false
  render: (data) =>
    inLi = (text) -> "<li>#{text}</li>"
    if data isnt ''
      comments = JSON.parse data
      if comments.map?
        formatted = comments.map(inLi).join ''
        @out.innerHTML = "<ul>#{formatted}</ul>"

exports.Comments = Comments
```